Configuration Defaults

The existing CTS codebase has approximately 3,000 hardcoded values spread across stored procedures, application configuration files, and database rows in a loosely structured defaults table. These are not the same as LaunchDarkly feature flags and must not be conflated with them.

Feature Flags (LaunchDarkly)	CnpDefaults (this document)	Workflow Overrides (TenantTransitionOverrides)
Is SSFO enabledfor this tenant? Is Session Replay on? ~47 flags · on/offModule-level gating	Late claim cutoff = 60 days Capital expense threshold = $5,000 DC match confidence min = 85 ~3,000 values · typed scalarsFine-grained operational config	State specific auto-declines State specific late claims State specific DirectVer Per-state process variationState machine rules as data

CnpDefaults are not feature flags A feature flag answers "does this feature exist for this tenant?" CnpDefaults answer "what value does this feature use?" Whether the late claim cutoff is 60 days or 90 days is not a feature flag question. Both states have late claim logic, they just use different thresholds. Putting thresholds in LaunchDarkly would bloat flag management and mix two distinct concerns in one tool.

Hierarchy

Every configuration value resolves through a three-layer hierarchy. The engine always returns the most specific value available, falling back up the chain if no override exists at a lower level.

Layer 1 — Platform Default
The baseline value that applies to all states, all programs.
Lives in cnp_global
Example: LateClaimCutoffDays = 6

Layer 2 — Program Override

Overrides the platform default for a specific program.

Lives in cnp_global

Example: SFSP.LateClaimCutoffDays = 30 (summer programs close faster)

if no program override exists, it moves up to layer 1 →

Layer 3 — Tenant Override

Overrides for a specific state.

Lives in cnp_ops.

Cannot exceed ValidationBounds.

Example: ME.LateClaimCutoffDays = 45 (Maine negotiated a different window)

if no tenant override exists, it moves up to layer 2 →

Resolution example

DC match confidence threshold for Utah SFSP

```
-- Engine resolves: ConfigKey='DCMatchConfidenceMin', Program='SFSP', Tenant='UT'\n\nSELECT COALESCE(\n  -- Layer 3: Tenant+Program specific\n  (SELECT Value FROM TenantConfigOverrides\n   WHERE TenantCode='UT' AND ProgramCode='SFSP' AND ConfigKey='DCMatchConfidenceMin'\n   AND EffectiveDate <= TODAY() ORDER BY EffectiveDate DESC LIMIT 1),\n  -- Layer 3: Tenant-wide\n  (SELECT Value FROM TenantConfigOverrides\n   WHERE TenantCode='UT' AND ProgramCode IS NULL AND ConfigKey='DCMatchConfidenceMin'\n   AND EffectiveDate <= TODAY() ORDER BY EffectiveDate DESC LIMIT 1),\n  -- Layer 2: Program default\n  (SELECT DefaultValue FROM ConfigDefinitions\n   WHERE ProgramCode='SFSP' AND ConfigKey='DCMatchConfidenceMin'),\n  -- Layer 1: Platform default\n  (SELECT DefaultValue FROM ConfigDefinitions\n   WHERE ProgramCode IS NULL AND ConfigKey='DCMatchConfidenceMin')\n) AS ResolvedValue\n-- Returns: 85 (platform default, no overrides exist for UT/SFSP)
```

Tables

ConfigDefinitions (cnp_global)

The master catalog of every configuration key the platform knows about. One row per key per program (null program = platform-wide). This table is what transforms "3,000 hardcoded values in stored procedures" into "3,000 documented, typed, bounded configuration keys."

Column	Type	Notes
ConfigDefId	PK	

ConfigKey	varchar	Human-readable dotted key: `Claims.LateClaimCutoffDays`, `DC.MatchConfidenceMin`
ProgramCode	varchar null	Null = platform-wide default; 'NSLP', 'SFSP', 'CACFP' = program-specific
ValueType	varchar	'int', 'decimal', 'bool', 'string', 'date'
DefaultValue	varchar	Stored as string, cast to ValueType at read time
MinValue	varchar null	Validation lower bound — engine rejects overrides below this
MaxValue	varchar null	Validation upper bound
AllowedValues	JSONB null	For enum-type configs: `["daily","weekly","monthly"]`
IsFederal	bool	True = tenant cannot override. Platform value only.
Category	varchar	'Claims', 'DirectCert', 'Compliance', 'Notifications', etc. — for admin UI grouping
Description	text	Plain-English explanation of what this value controls
FederalCitation	varchar null	CFR section if federally prescribed: `7 CFR 210.19(c)`

TenantConfigOverrides (cnp_ops)

Per-state overrides of platform or program defaults. One row per override. Effective-dated so changes take effect on a specific date without requiring a deployment.

Column	Type	Notes
OverrideId	PK	
ConfigDefId	FK → ConfigDefinitions	
TenantCode	varchar	State identifier
ProgramCode	varchar null	Null = applies to all programs for this tenant

OverrideValue	varchar	The state-specific value. Engine validates against MinValue/MaxValue/AllowedValues.
EffectiveDate	date	Value takes effect on this date. Can be future-dated.
ExpiresDate	date null	Optional expiration. Null = no expiration.
SetBy	FK → WebUsers	Onshore config team member who set this value
SetAt	timestamp	
Justification	text	Required field — reason for the override, for audit trail

IsFederal blocks tenant overrides at write time When a TenantConfigOverride is written, the engine checks ConfigDefinitions.IsFederal. If true, the write is rejected. Example: `CEP.ISPMultiplier` (the 1.6 factor) has IsFederal=true, FederalCitation='42 USC 1759a', MinValue=1.6, MaxValue=1.6. No state can change it. No UI exposes it for editing.

Configuration Categories

The 3,000+ keys group into eight categories. The category drives admin UI organization and access control — a state nutrition director should see Claims and Compliance config; they should not see infrastructure config.

Claims

- LateClaimCutoffDays
- MaxMealsPerStudentPerDay
- ClaimRevisionWindowDays
- AutoHoldThresholdPct (% over prior month triggers review)
- BatchAcceptanceLockoutHours

Direct Certification

- DCMatchConfidenceMin (85)
- DCPossibleMatchMin (50)
- DCLoadRetryAttempts
- DCLoadAlertLagMinutes
- DCFeedStaleAfterHours

Compliance

- ReviewCycleYears (3)
- HighRiskReviewThreshold (score)
- FiveDaySampleDaysRequired (5)
- FiscalActionAnomalyPctAlert (10%)
- SeriousDeficiencyGraceDays

Applications

- ApplicationRenewalWindowDays

Notifications

- ClaimDeadlineReminderDays (7, 3, 1)

Reporting

- FNSSubmissionWindowDays

- PotentialSponsorExpiryDays
- VCAPassingScoreThreshold
- CapitalExpenditureApprovalMin ($5,000)
- FSMCContractMaxYears
- ReviewOverdueAlertDays
- DCLoadFailureAlertDelayMin
- ApplicationExpiryWarnDays
- DisburseDelayAlertDays
- SnowflakeCDCLagAlertMinutes
- ReportAnomalyVariancePct
- FNS10Section4PctThreshold (UT)
- AnnualRateAnomalyMaxPct (10%)

Program-Specific

- SFSP.NonCongregateMealPickupWindowHrs
- CACFP.MealLimitPerChildPerDay (2 meals + 1 snack)
- CEP.ISPMultiplier (1.6 — federal, locked)
- FFVP.UnexpendedBalanceAlertPct
- DCH.TierINeighborhoodPovPct (50%)

Infrastructure

- SessionTimeoutMinutes
- APIRateLimitPerMinute
- MaxFileUploadSizeMB
- PasswordExpiryDays
- AuditLogRetentionDays

Effective Dating

Configuration values often need to change on a specific future date (e.g. a new state contract takes effect July 1, a regulatory threshold changes at the start of the program year, a pilot change needs to be reversed if it causes problems). Effective dating lets onshore teams set these changes in advance without a deployment window.

Example

Minnesota contract renegotiation takes effect July 1, 2026

Minnesota's new contract changes their late claim cutoff from 60 days to 45 days. The onshore config team sets this in April before the contract is signed, effective July 1:

```
-- Written in April; takes effect July 1 without any deployment\nTenantConfigOverrides:\n  ConfigKey     = 'Claims.LateClaimCutoffDays'\n  TenantCode    = 'MN'\n  OverrideValue = '45'\n  EffectiveDate = 2026-07-01\n  Justification = 'Minnesota SY2026-27 contract amendment, section 4.2'
```

Between now and June 30, the engine resolves Minnesota's cutoff as 60 (platform default). From July 1 onward, it resolves as 45. No deployment. No stored procedure change. The old value remains in the table as history.

The resolution query always filters `EffectiveDate <= TODAY()` and returns the most recent matching row. This means future-dated overrides sit in the table harmlessly until their effective date arrives — they don't require a scheduled job to activate, they just become the winning row in the COALESCE chain.

Validation Bounds

Every ConfigDefinition row has optional MinValue, MaxValue, and AllowedValues columns. The engine validates any override attempt against these bounds before writing the row. This prevents misconfiguration — a typo setting the DC confidence threshold to 8 instead of 80, or the late claim cutoff to 600 days instead of 60.

ConfigKey	Type	Default	Min	Max	IsFederal	Citation
`DC.MatchConfidenceMin`	int	85	50	99	No	—
`Claims.LateClaimCutoffDays`	int	60	1	60	Yes	—
`Compliance.ReviewCycleYears`	int	3	1	3	Yes	7 CFR 210.18(b)
`CEP.ISPMultiplier`	decimal	1.6	1.6	1.6	Yes	42 USC 1759a
`CACFP.MealLimitPerChildPerDay`	int	3	3	3	Yes	7 CFR 226.20
`Apps.CapitalExpenditureMin`	decimal	5000	0	25000	No	—
`Notifications.ClaimReminderDays`	string	7,3,1	—	—	No	—

Federal values with Min = Max = Default

For federally locked values like the CEP ISP multiplier (1.6) and the CACFP meal limit (3), MinValue and MaxValue are both set equal to DefaultValue. This makes it mathematically impossible to set a different value even for users who have override permissions — the validation rejects anything outside [1.6, 1.6]. The IsFederal flag is the belt; the bounds are the suspenders.

Resolution at Runtime

The engine exposes a single typed method for consuming any config value. All platform code calls this method. Nothing reads the raw tables directly outside of the config service itself.

```
// C# typed config resolution — platform code never reads ConfigDefinitions directly\npublic interface ICnpConfigService {\n  T GetValue<T>(string configKey, string tenantCode, string? programCode = null);\n}\n\n// Usage examples throughout the platform:\nvar cutoffDays = _config.GetValue<int>("Claims.LateClaimCutoffDays", tenantCode, "NSLP");\nvar matchMin  = _config.GetValue<int>("DC.MatchConfidenceMin", tenantCode);\nvar ispMult   = _config.GetValue<decimal>("CEP.ISPMultiplier", tenantCode); // always 1.6
```

> ⚠ We must prohibit reading ConfigDefinitions or TenantConfigOverrides directly in application code. The ICnpConfigService interface is the only sanctioned path. This ensures the three-layer resolution logic, bounds validation, and caching are always applied so there is no accidental hardcoding of the default value in a service that forgot to call the config layer.

Cache

Tier 1: Immediate (5-minute TTL)

Config keys where a stale value could cause a real-time compliance or eligibility error.

Example key	Why it matters
`DCMatch.ConfidenceMinimum`	Affects live eligibility determinations — serving the wrong threshold can incorrectly approve or deny a direct certification match
`Claims.FnsSubmissionDeadlineDays`	Affects whether a claim submission is accepted or rejected at the moment it is submitted
`NDL.OverrideRequiresApproval`	Affects whether the NDL block is enforced in real time — a stale false value would pass through a blocked record

Redis with 5-minute TTL. Invalidated immediately when a new override is saved for this key.

Tier 2:Session (per-request cache)

Config keys used for display, form rendering, and UI defaults where a same-request value is required for consistency but a slightly stale value between requests.

Example key	Why it matters
UI labels, display thresholds, notification content templates	Must be consistent within a single rendered page; cross-request staleness is harmless
`Claims.InternalClaimCutoffDays`	Affects form validation but not real-time processing — a slightly stale value between requests causes no compliance risk

Scoped to the request context (`IMemoryCache` , per-request scope). No shared cache.

Tier 3: Nightly (midnight UTC refresh)

Config keys that change annually or semi-annually and where same-day changes are operationally impossible under normal circumstances.

Example key	Why it matters
Reimbursement rates (annual FNS updates)	Updated once per year when FNS publishes new rates; serving yesterday's rate for a few hours is inconsequential
Program year start/end dates	Fixed at program year open; never change mid-year
Poverty guideline thresholds	Published annually by HHS; same-day change is never required

Loaded at startup, refreshed via Hangfire job at midnight UTC. The midnight job picks up any Pre-staged effective-date overrides when `EffectiveDate` has arrived.

Cache tier assignment

Each `ConfigDefinitions` row has a `CacheTier` column with values `Immediate` , `Session` , or `Nightly` . The default is `Session` . Tier assignment is set at key definition time by LINQ engineering — it is not configurable by state administrators.

Manual cache bust

`POST /admin/config/cache/bust?tenantCode={code}&configKey={key}`

Rule	Detail

Authorization	Requires `LINQConfigAdmin` role
Effect	Immediately invalidates the Redis cache entry for that key + tenant combination
Audit trail	Every call logged to `AuditLogs` with `EntityType = 'ConfigCacheBust'`, `ActorUserId`, `TenantCode`, `ConfigKey`, `Timestamp`
Response	Confirms the new value that will be served on the next read
Rate limit	Max 10 cache bust operations per admin per hour — prevents cache stampede

Example

An onshore config admin sets an emergency override for a Tier 1 key at 2pm.

1. Override written to TenantConfigOverrides
   a. (EffectiveFrom = now; immediately active per effective-date logic)
2. Write service calls the cache invalidation endpoint automatically
3. Redis entry expires; next request reads the new value from the database

Total propagation time: under 5 minutes

> ⚠ For Tier 3 keys, an emergency override set at 2pm will **not** take effect until midnight or until a manual cache bust is issued.

Admin UI

The configuration admin UI is the primary surface onshore teams use to manage per-state values. It is structured around the eight categories and surfaced in the Configuration & Tenant domain of the admin shell.

The admin UI enforces bounds visually — a numeric field for `Claims.LateClaimCutoffDays` shows a range indicator (1–90) and rejects out-of-range input before the API is called. Federal rows render as read-only with a lock icon and citation

reference. All writes require a Justification field and write to FieldChangeLog. The History button shows the full audit trail for every override ever set on this tenant.

Schema Gap: Per-State Error Configuration — Added July 2026

Identified during review of CTS stored procedures (June 30, 2026) and added to CNP_ERD.html. The global Errors catalog lives in cnp_global. There was no per-state configuration layer for error descriptions, severity overrides, or effective dates. Austin confirmed: states can change error descriptions, flip error to warning, and set start/end dates. TenantErrorConfig is the cnp_ops companion table. Also see: **StatusTransitionRules** on the Workflow Engine page.

Errors cnp_global

The global validation error catalog. Every error the platform can produce has one row here. States read from this catalog; they cannot modify it directly. Their per-state customizations live in TenantErrorConfig (cnp_ops).

Column	Notes
ErrorId	PK
ErrorCode	Machine-readable code: CLAIM_MEAL_COUNT_EXCEEDS_ENROLLMENT
DefaultDescription	Platform default message shown to users when no state override exists
DefaultSeverity	Error | Warning — platform default
ProgramCode	null = cross-program. NSLP, CACFP, etc. = program-specific
IsActive	Soft-delete
IsFederal	True = states cannot override severity (error is federally required). Description overrides still allowed.

TenantErrorConfig cnp_ops

Per-state overrides of the global error catalog. Null fields inherit from the global default. This is the greenfield replacement for per-state error configuration logic scattered across CTS stored procedures.

Column	Notes
TenantErrorConfigId	PK
TenantCode	FK → tenant
ErrorId	FK → cnp_global.Errors
OverrideDescription	null = use global default description
OverrideType	Error | Warning | null (null = use global default severity). IsFederal=true errors block severity downgrade — engine rejects Error to Warning overrides on federal errors.
StartDate	null = always active. Non-null = override takes effect on this date
EndDate	null = perpetual. Non-null = override expires on this date
ModifiedBy	FK → WebUsers — onshore team member who set this
ModifiedAt	timestamp

⚠ TenantErrorConfig cannot downgrade a federal error to a warning. The write service checks Errors.IsFederal before accepting an OverrideType = Warning write. Any attempt is rejected and logged to AuditLogs with EntityType = FederalGuardrailViolation.